

LABSHEET 10

NAME : KANISHKA SHARMA

CUID : CU24250085

ROLLNO : 26

COURSE : BTECH CSE 'B'

```
# Import required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import chi2_contingency, ttest_ind

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

import warnings
warnings.filterwarnings('ignore')

sns.set_style("whitegrid")

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose files BANK.DATASET.csv
 BANK.DATASET.csv(text/csv) - 884858bytes, last modified: 21/09/2028 - 100% done
 Saving BANK.DATASET.csv to BANK.DATASET (1).csv

```
import io

filename = list(uploaded.keys())[0]

df = pd.read_csv(io.BytesIO(uploaded[filename]))

print("Dataset loaded successfully!")
print("Dataset Shape:", df.shape)

df.head()
```

Dataset loaded successfully!
 Dataset Shape: (18000, 14)

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	1	15634602 Hargrave	619	France	Female	42	2	0.00	1	1	1	
1	2	15647311 Hill	808	Spain	Female	41	1	83807.88	1	0	1	
2	3	15619304 Onio	502	France	Female	42	8	159860.80	3	1	0	
3	4	15701354 Boni	809	France	Female	39	1	0.00	2	0	0	
4	5	15737888 Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Display basic information about the dataset

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()
```

Number of Rows: 18000
 Number of Columns: 14

Column Names:
 ['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary']

```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  --
---  ---
0   RowNumber              10000 non-null  int64
1   CustomerId             10000 non-null  int64
2   Surname                10000 non-null  object
3   CreditScore             10000 non-null  int64
4   Geography              10000 non-null  object
5   Gender                 10000 non-null  object
6   Age                    10000 non-null  int64
7   Tenure                 10000 non-null  int64
8   Balance                10000 non-null  float64
9   NumOfProducts          10000 non-null  int64
10  HasCrCard              10000 non-null  int64
11  IsActiveMember         10000 non-null  int64
12  EstimatedSalary        10000 non-null  float64
13  Exited                 10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB

```

```
# Statistical summary of numerical columns
```

```
df.describe()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
count	10000.00000	10000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000	10000.000000
mean	5000.50000	1.589094e+07	850.528800	38.921800	5.012800	76486.889288	1.530200	0.70550	0.515100	10000.000000
std	2886.89568	7.193019e+04	98.853299	10.487806	2.892174	82397.405202	0.581654	0.45584	0.499797	575.000000
min	1.00000	1.558570e+07	360.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000	0.000000
25%	2500.75000	1.582853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000	0.000000
50%	5000.50000	1.589074e+07	852.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000	510.000000
75%	7500.25000	1.576323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000	10010.000000
max	10000.00000	1.581589e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000	14938.000000

```
# Check duplicate rows
```

```
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

```
# Remove duplicate rows
```

```
df = df.drop_duplicates()
```

```
print("Duplicates removed successfully!")
print("New dataset shape:", df.shape)
```

```
Duplicates removed successfully!
New dataset shape: (10000, 14)
```

```
# Check the target variable
```

```

if 'Exited' in df.columns:
    print(df['Exited'].value_counts())
else:
    print("Target column 'Exited' not found.")

```

```

Exited
0    7963
1    2037
Name: count, dtype: int64

```

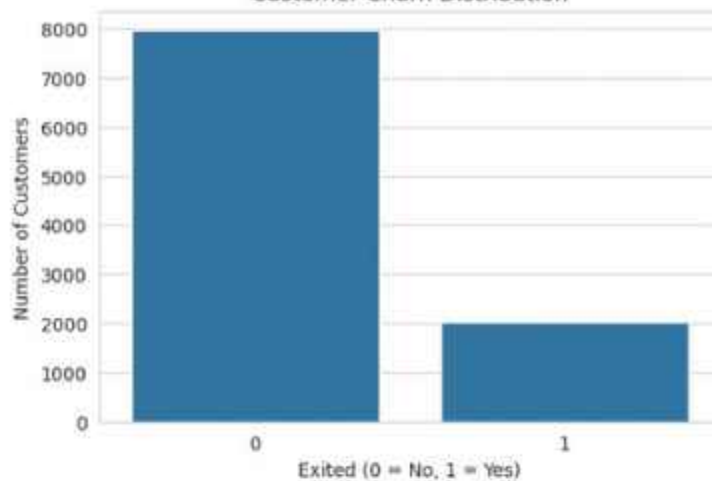
```
# Visualize customer churn distribution
```

```

if 'Exited' in df.columns:
    plt.figure(figsize=(6,4))
    sns.countplot(x='Exited', data=df)
    plt.title("Customer Churn Distribution")
    plt.xlabel("Exited (0 = No, 1 = Yes)")
    plt.ylabel("Number of Customers")
    plt.show()

```

Customer Churn Distribution



```
# Identify numerical and categorical columns
```

```
numerical_columns = df.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df.select_dtypes(include='object').columns.tolist()
```

```
print("Numerical Columns:")
print(numerical_columns)
```

```
print("\nCategorical Columns:")
print(categorical_columns)
```

```
Numerical Columns:
['RowNumber', 'CustomerId', 'CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary',
Categorical Columns:
['Surname', 'Geography', 'Gender']
```

```
# Handle missing values
```

```
for column in df.columns:
    if df[column].isnull().sum() > 0:
        if df[column].dtype == 'object':
            df[column] = df[column].fillna(df[column].mode()[0])
        else:
            df[column] = df[column].fillna(df[column].median())
```

```
print("Missing values after preprocessing:")
print(df.isnull().sum())
```

```
Missing values after preprocessing:
RowNumber      0
CustomerId      0
Surname         0
CreditScore     0
Geography       0
Gender          0
Age             0
Tenure          0
Balance         0
NumOfProducts   0
HasCrCard       0
IsActiveMember  0
EstimatedSalary 0
Exited          0
dtype: int64
```

```
# Remove unnecessary identifier columns
```

```
columns_to_remove = ['RowNumber', 'CustomerId', 'Surname']
```

```
for column in columns_to_remove:
    if column in df.columns:
        df.drop(column, axis=1, inplace=True)
```

```
print("Unnecessary columns removed.")
print(df.head())
```

```
Unnecessary columns removed.
   CreditScore Geography Gender  Age  Tenure  Balance  NumOfProducts  \
0         619   France  Female   42     2     0.00             1
1         608    Spain  Female   41     1  83807.06             1
2         502   France  Female   42     8  159668.00             3
3         699   France  Female   39     1     0.00             2
4         858    Spain  Female   43     2  125518.62             1
```

```
HasCrCard  IsActiveMember  EstimatedSalary  Exited
```

0	1	1	101348.88	1
1	0	1	112542.58	0
2	1	0	113931.57	1
3	0	0	93826.63	0
4	1	1	79084.10	0

```
# Convert categorical variables into numerical form

label_encoder = LabelEncoder()

for column in df.select_dtypes(include='object').columns:
    df[column] = label_encoder.fit_transform(df[column])

print("Categorical variables encoded successfully!")
df.head()
```

Categorical variables encoded successfully!

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited	
0	619		0	0	42	2	0.00	1	1	1	101348.88	1
1	608		2	0	41	1	83807.80	1	0	1	112542.58	0
2	502		0	0	42	0	159660.80	3	1	0	113931.57	1
3	590		0	0	39	1	0.00	2	0	0	93826.63	0
4	550		2	0	43	2	125510.82	1	1	1	79084.10	0

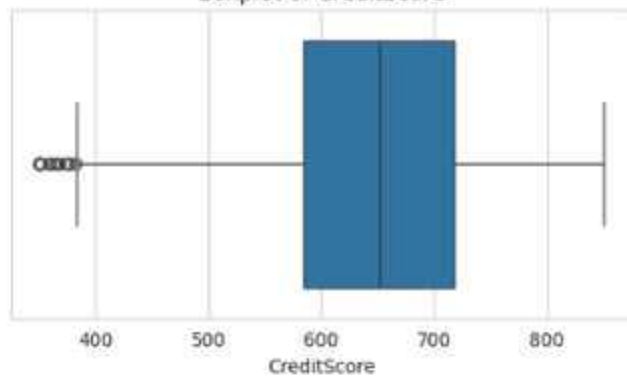
Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Check outliers using boxplots

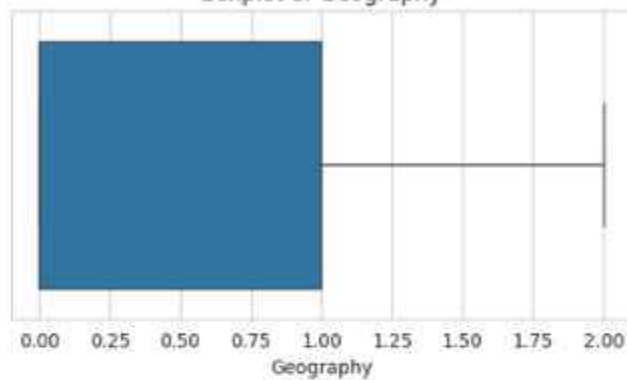
numeric_cols = df.select_dtypes(include=np.number).columns

for column in numeric_cols:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[column])
    plt.title("Boxplot of " + column)
    plt.show()
```

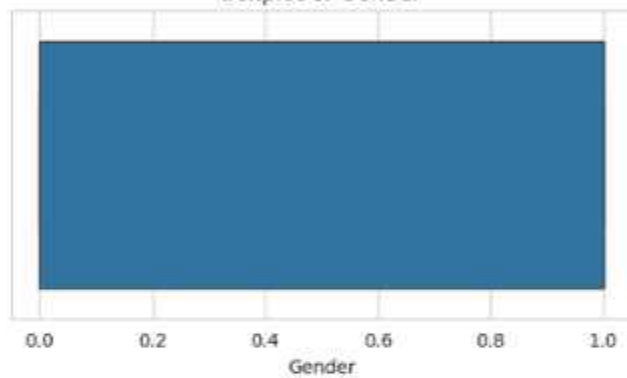
Boxplot of CreditScore



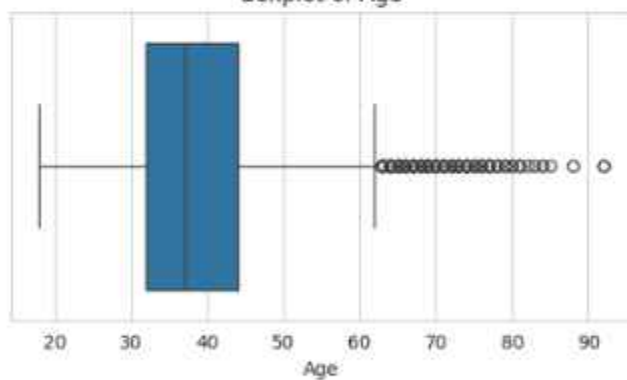
Boxplot of Geography



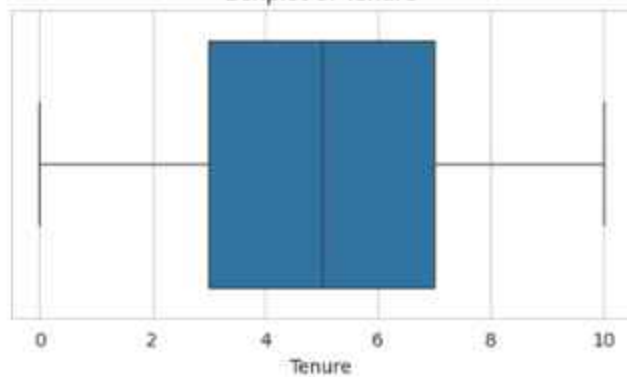
Boxplot of Gender



Boxplot of Age

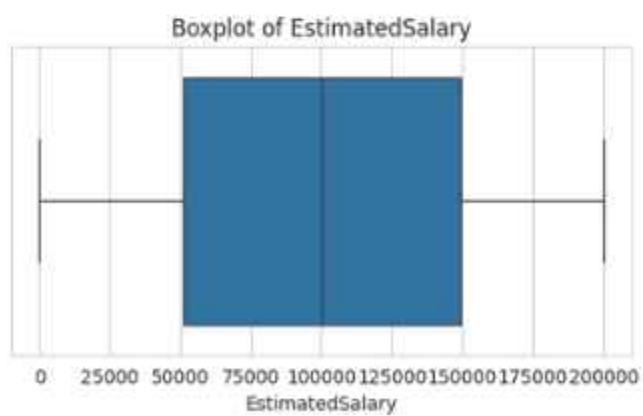
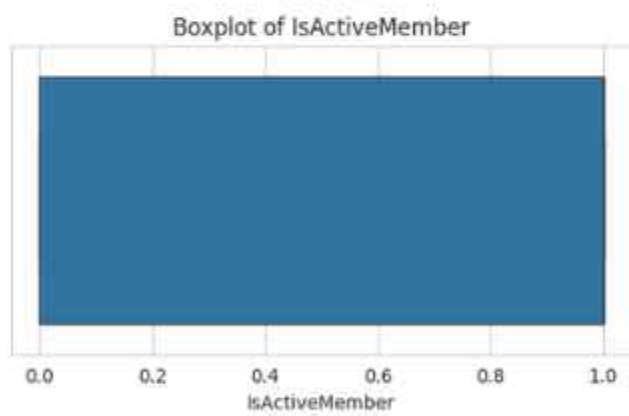
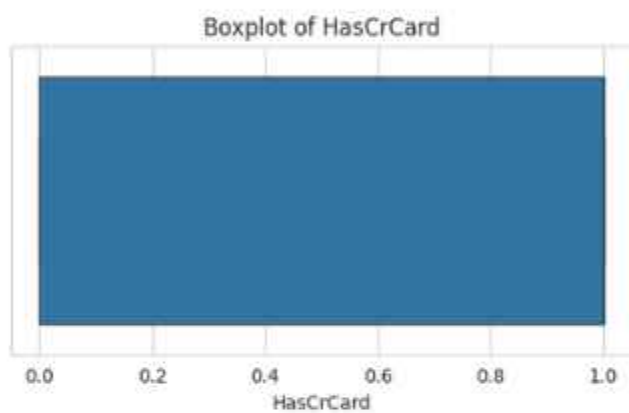
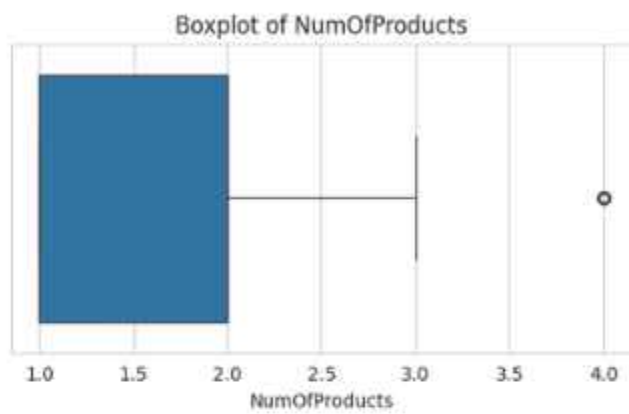
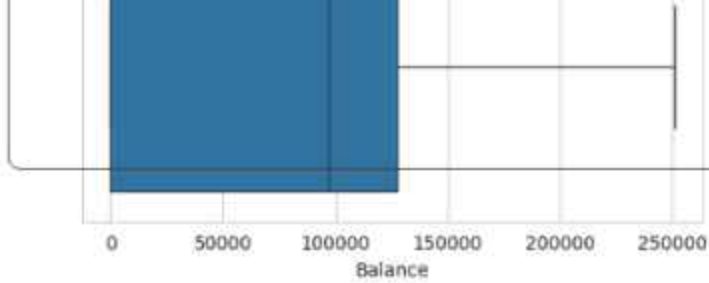


Boxplot of Tenure



Boxplot of Balance





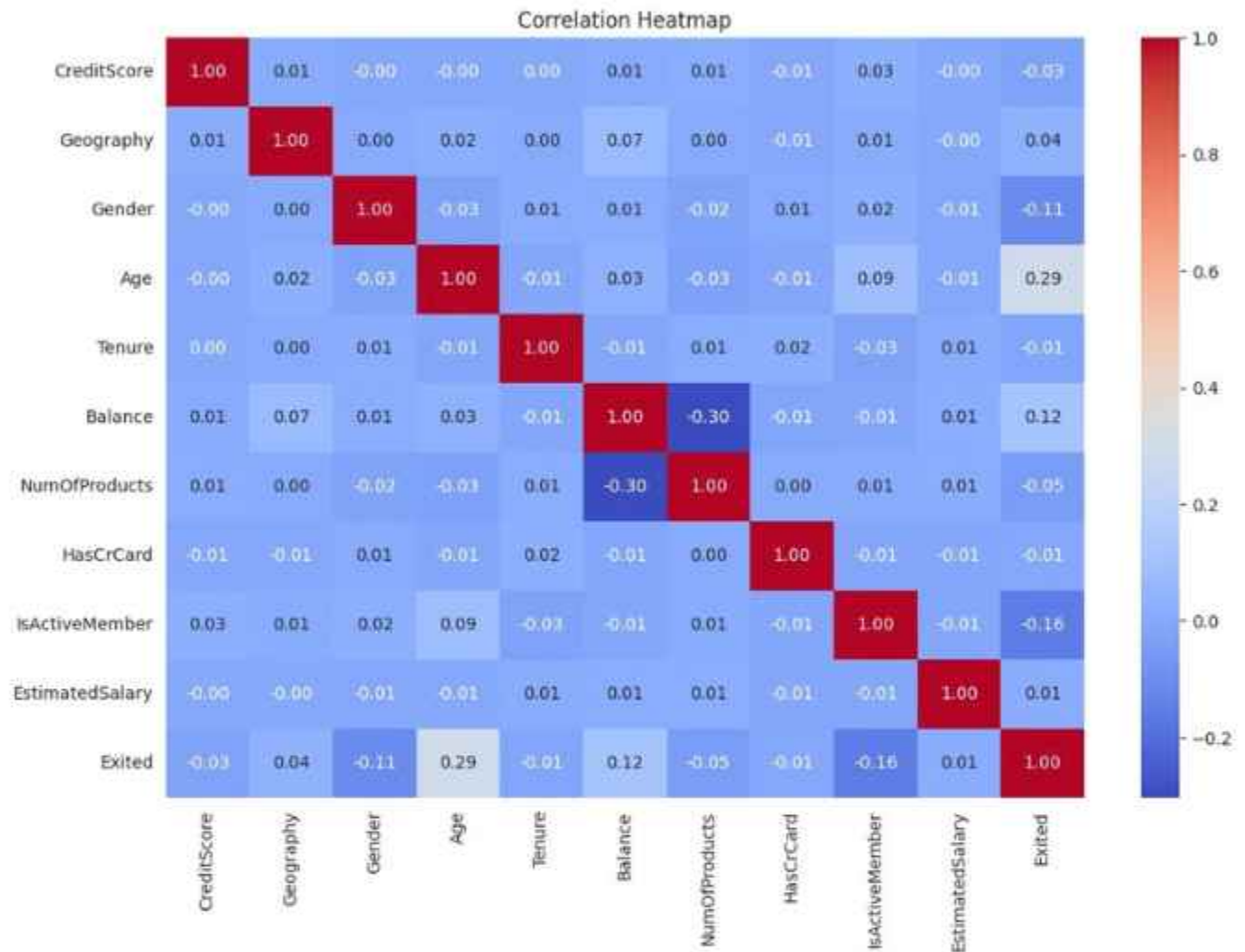
```
# Correlation analysis

plt.figure(figsize=(12,8))

correlation = df.corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Heatmap")
plt.show()
```



```
# Analyse relationship between age and churn

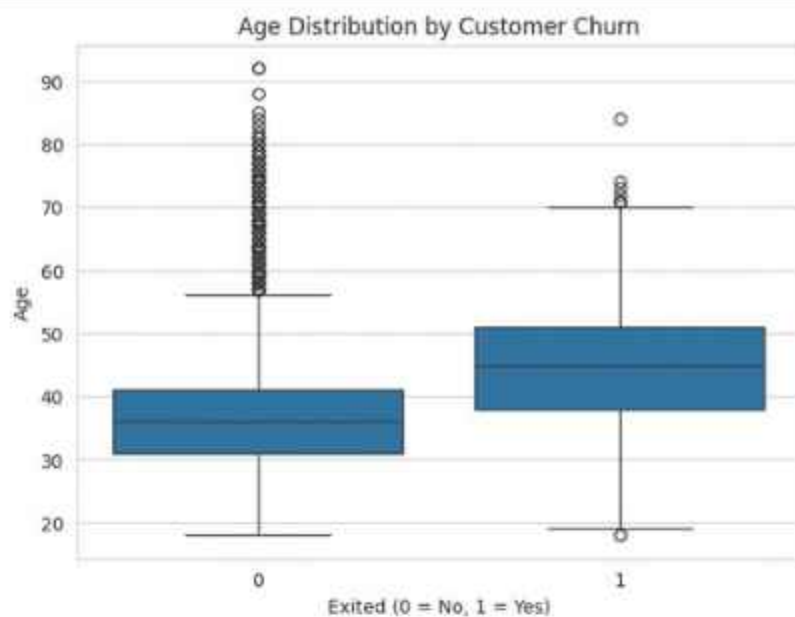
if 'Age' in df.columns and 'Exited' in df.columns:

    plt.figure(figsize=(7,5))

    sns.boxplot(
        x='Exited',
        y='Age',
        data=df
    )

    plt.title("Age Distribution by Customer Churn")
    plt.xlabel("Exited (0 = No, 1 = Yes)")
    plt.ylabel("Age")

    plt.show()
```



```
# Analyze balance and churn

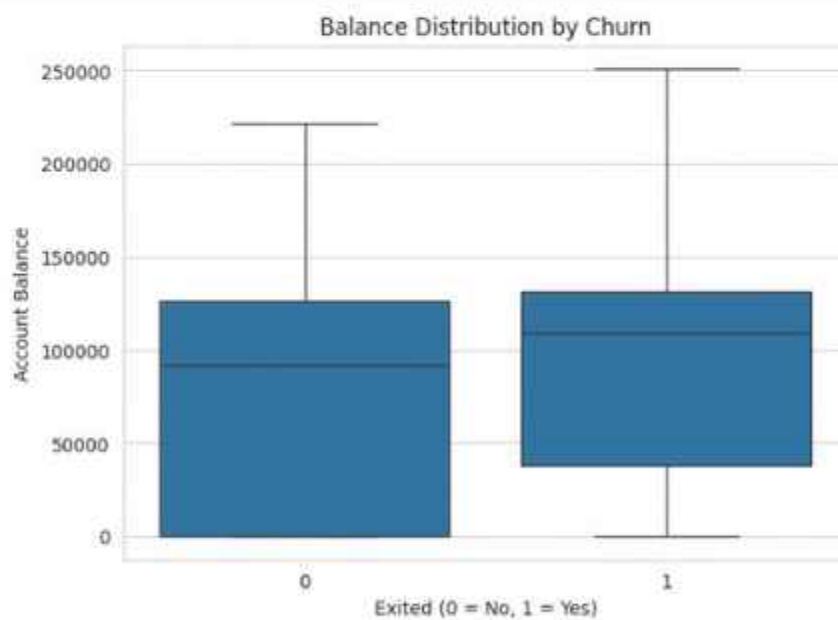
if 'Balance' in df.columns and 'Exited' in df.columns:

    plt.figure(figsize=(7,5))

    sns.boxplot(
        x='Exited',
        y='Balance',
        data=df
    )

    plt.title("Balance Distribution by Churn")
    plt.xlabel("Exited (0 = No, 1 = Yes)")
    plt.ylabel("Account Balance")

    plt.show()
```



```
# Geography and churn analysis
```

```
if 'Geography' in df.columns and 'Exited' in df.columns:
```

```
plt.figure(figsize=(7,5))
```

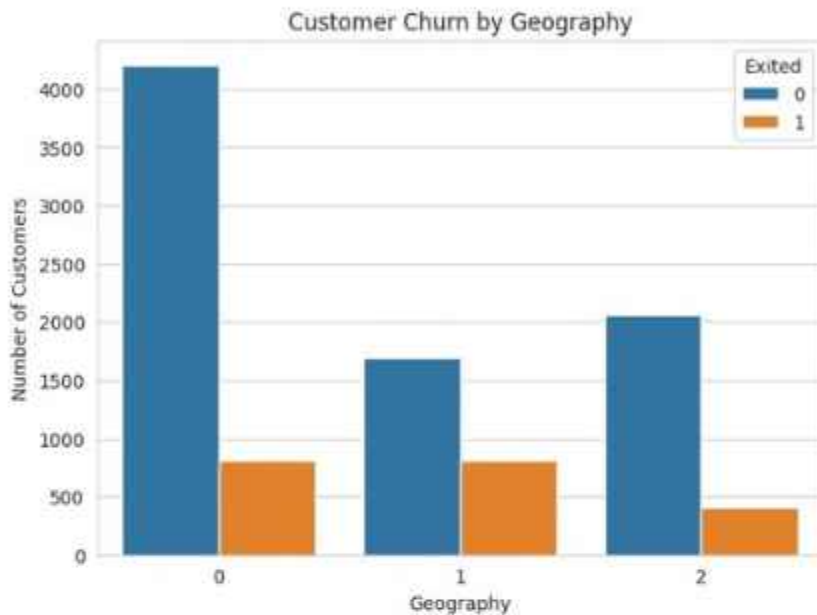
```
sns.countplot(
    x='Geography',
    hue='Exited',
    data=df
)
```

```
plt.title("Customer Churn by Geography")
```

```
plt.xlabel("Geography")
```

```
plt.ylabel("Number of Customers")
```

```
plt.show()
```



```
# Gender and churn analysis
```

```
if 'Gender' in df.columns and 'Exited' in df.columns:
```

```
plt.figure(figsize=(7,5))
```

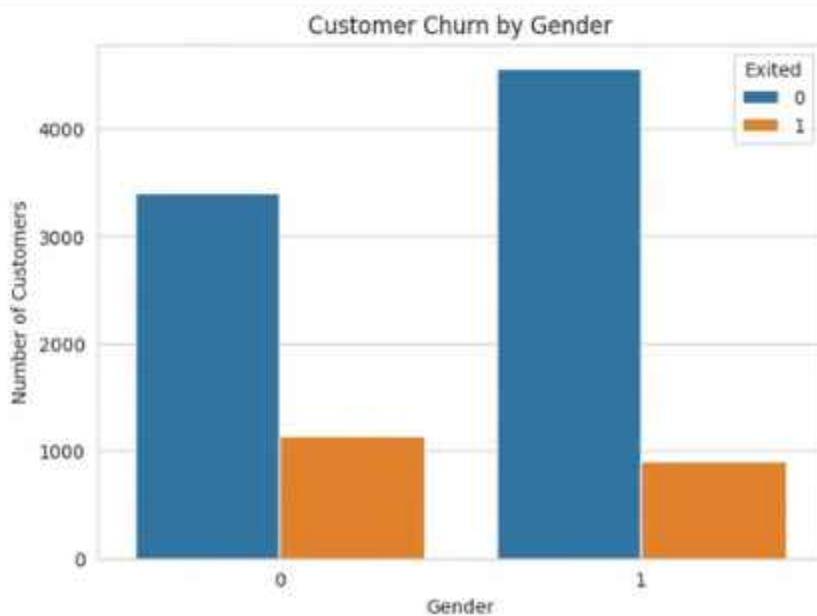
```
sns.countplot(
    x='Gender',
    hue='Exited',
    data=df
)
```

```
plt.title("Customer Churn by Gender")
```

```
plt.xlabel("Gender")
```

```
plt.ylabel("Number of Customers")
```

```
plt.show()
```



```
# Tenure and churn analysis
```

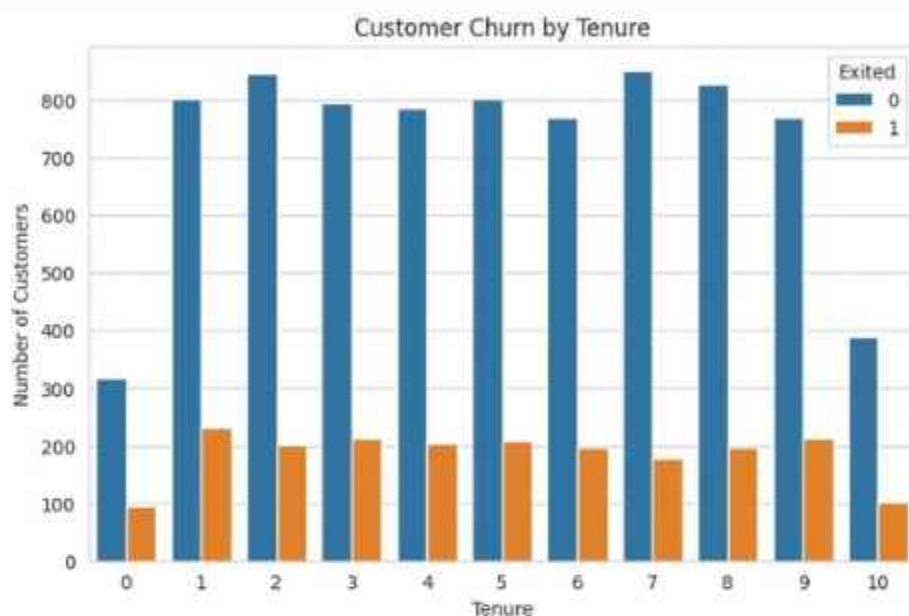
```
if 'Tenure' in df.columns and 'Exited' in df.columns:
```

```
plt.figure(figsize=(8,5))
```

```
sns.countplot(
    x='Tenure',
    hue='Exited',
    data=df
)
```

```
plt.title("Customer Churn by Tenure")
plt.xlabel("Tenure")
plt.ylabel("Number of Customers")
```

```
plt.show()
```



```
# T-test between age groups of churned and non-churned customers
```

```
if 'Age' in df.columns and 'Exited' in df.columns:
```

```
churned_age = df[df['Exited'] == 1]['Age']
retained_age = df[df['Exited'] == 0]['Age']
```

```
t_stat, p_value = test_ind(churned_age, retained_age)
```

```
print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
if p_value < 0.05:
```

```
    print("There is a statistically significant difference in age between churned and retained customers.")
```

```
else:
```

```
    print("There is no statistically significant difference in age between churned and retained customers.")
```

```
T-statistic: 29.76681499437877
```

```
P-value: 1.2399313893427736e-186
```

```
There is a statistically significant difference in age between churned and retained customers.
```

```
# Chi-square test between Geography and Churn
```

```
if 'Geography' in df.columns and 'Exited' in df.columns:
```

```
contingency_table = pd.crosstab(
    df['Geography'],
    df['Exited']
)
```

```
chi2, p, dof, expected = chi2_contingency(contingency_table)
```

```
print("Chi-square statistic:", chi2)
print("P-value:", p)
```

```
if p < 0.05:
```

```
    print("Geography and customer churn have a statistically significant association.")
```

```
else:
```

```
    print("There is no statistically significant association between Geography and churn.")
```

```
Chi-square statistic: 381.25533682434536
```

```
P-value: 3.8383176853541544e-66
```

```
Geography and customer churn have a statistically significant association.
```

```
# Separate features and target variable
```

```
X = df.drop('Exited', axis=1)
y = df['Exited']
```

```
print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (18000, 10)
Target shape: (18000,)
```

```
# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (8000, 10)
Testing data: (2000, 10)
```

```
# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (8000, 10)
Testing data: (2000, 10)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Create and train Logistic Regression model
```

```
model = LogisticRegression(random_state=42)

model.fit(X_train_scaled, y_train)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

```
# Create and train Logistic Regression model
```

```
model = LogisticRegression(random_state=42)

model.fit(X_train_scaled, y_train)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

```
# Make predictions on test data
```

```
y_pred = model.predict(X_test_scaled)

print("Predictions generated successfully!")
print(y_pred[:10])
```

```
Predictions generated successfully!
[0 0 0 0 0 0 0 0 0 0 1 0]
```

```
# Calculate model performance metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print("Model Evaluation")
```

```
print("-----")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))
```

Model Evaluation

```
-----
Accuracy : 0.885
Precision: 0.5859
Recall   : 0.9425
F1 Score : 0.7292
```

Confusion Matrix

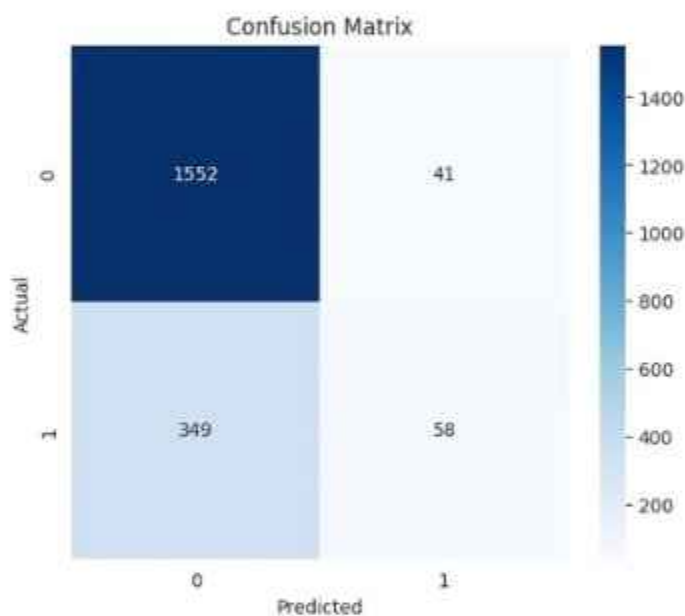
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



Detailed classification report

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.82	0.97	0.89	1593
1	0.59	0.14	0.23	487
accuracy			0.81	2080
macro avg	0.70	0.56	0.56	2080
weighted avg	0.77	0.81	0.75	2080

Analyze Logistic Regression coefficients

```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

feature_importance['Absolute_Coefficient'] = abs(
    feature_importance['Coefficient']
)

feature_importance = feature_importance.sort_values(
    by='Absolute_Coefficient',
    ascending=False
)
```

	Feature	Coefficient	Absolute_Coefficient
3	Age	0.739482	0.739482
8	IsActiveMember	-0.519648	0.519648
5	Balance	0.314425	0.314425
2	Gender	-0.267857	0.267857
8	CreditScore	-0.086363	0.086363
1	Geography	0.075674	0.075674
9	EstimatedSalary	0.049585	0.049585
6	NumOfProducts	-0.031184	0.031184
7	HasCrCard	-0.025533	0.025533
4	Tenure	-0.017044	0.017044

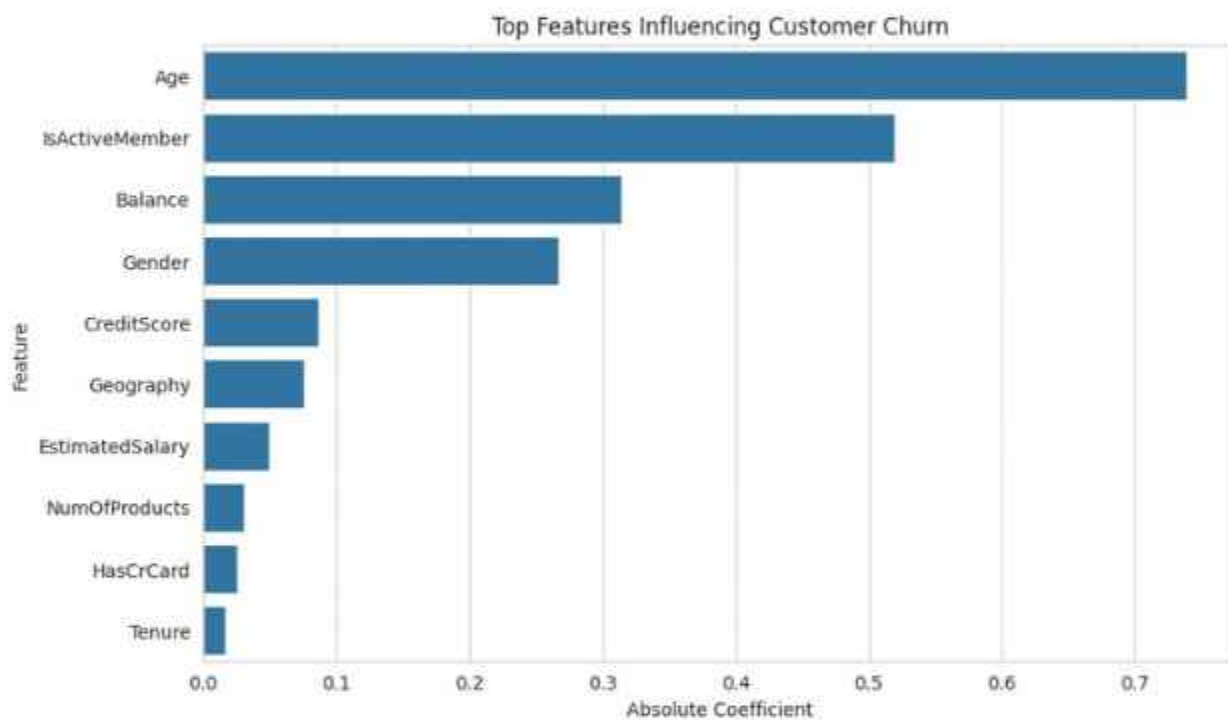
```
# Visualize important features

plt.figure(figsize=(10,6))

sns.barplot(
    x='Absolute_Coefficient',
    y='Feature',
    data=feature_importance.head(10)
)

plt.title("Top Features Influencing Customer Churn")
plt.xlabel("Absolute Coefficient")
plt.ylabel("Feature")

plt.show()
```



```
# Calculate overall churn rate

print("Overall Customer Churn Rate:", round(churn_rate, 2), "%")

Overall Customer Churn Rate: 20.37 %
```

End-to-End Data Analytics Project: Business Intelligence and Predictive Analytics

1. What are the major stages involved in an end-to-end data analytics project?

Answer: The major stages involved in an end-to-end data analytics project are data collection, data preprocessing, exploratory data analysis, statistical analysis, data visualization, predictive modeling, model evaluation, interpretation of results, and presentation of business insights. These stages help convert raw data into useful information for decision-making.

2. Why is data preprocessing considered the foundation of successful analytics and machine learning?

Answer: Data preprocessing is considered the foundation because real-world datasets may contain missing values, duplicate records, outliers, inconsistent formats, and categorical data. Preprocessing cleans and transforms the data into a suitable format for analysis and machine learning. Good-quality data improves the reliability and accuracy of the final results.

3. How does Exploratory Data Analysis (EDA) contribute to model development?

Answer: EDA helps understand the structure and characteristics of the dataset. It identifies important patterns, relationships, trends, outliers, and unusual values. EDA also helps determine which variables may be useful for prediction. Therefore, it supports better feature selection and helps in choosing an appropriate machine learning model.

4. What factors should be considered while selecting a machine learning algorithm for a business problem?

Answer: The selection of a machine learning algorithm depends on several factors such as the type of problem, size of the dataset, type of variables, target variable, relationship between features, model complexity, training time, interpretability, and required accuracy. For example, classification problems can use Logistic Regression or Decision Trees, while regression problems can use Linear Regression.

5. Why is model evaluation essential before deploying a predictive model?

Answer: Model evaluation is essential because a model may perform well on training data but poorly on unseen data. Evaluation metrics help measure the actual performance of the model. Metrics such as Accuracy, Precision, Recall, F1-Score, MAE, RMSE, and R^2 Score help determine whether the model is suitable for practical use.

6. Explain how Business Intelligence tools complement machine learning in data analytics projects.

Answer: Business Intelligence tools such as Power BI and Tableau help present analytical results through interactive dashboards, charts, and reports. Machine learning focuses on prediction and pattern detection, while Business Intelligence focuses on visualization, monitoring, and decision-making. Together, they provide both predictive insights and an easy way for business users to understand the results.

7. What challenges are commonly encountered while working with real-world datasets?

Answer: Common challenges include missing values, duplicate records, outliers, inconsistent data formats, incorrect values, categorical variables, imbalanced datasets, large data volumes, and noisy data. These problems need to be handled carefully because poor-quality data can affect the accuracy of analysis and machine learning models.

8. How can analytical insights be converted into actionable business recommendations?

Answer: Analytical insights can be converted into recommendations by identifying important patterns and understanding their business impact. For example, if analysis shows that certain customers have a higher probability of leaving, the company can provide them with personalized offers, better customer support, or customer retention programs. In this way, data-driven findings can support practical business decisions.

9. Why is effective presentation and data storytelling important in analytics projects?

Answer: Data storytelling makes complex analytical results easier to understand. Effective charts, dashboards, and explanations help decision-makers quickly identify important trends and patterns. A good presentation connects the analysis with the business problem and communicates the key findings and recommendations clearly.

10. Suggest future improvements or advanced techniques that could enhance the accuracy and effectiveness of the developed analytics solution.

Answer: Future improvements can include using advanced machine learning algorithms such as Random Forest, Gradient Boosting, XGBoost, or Neural Networks. Hyperparameter tuning and cross-validation can also be used to improve model performance. Feature engineering, handling class imbalance, advanced outlier detection, and deployment through an interactive Power BI dashboard or web application can further improve the analytics solution.